pfi

Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

Debugging

WEEK 1 · DAY 3

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

By the end of today you can

Read a traceback

The error type, the message, and the line.

```
NameError: name 'cupon' is not defined
```

Name the common errors

Each one points at a kind of mistake.

```
KeyError: 'coupon'
```

Catch a wrong number

The bugs that raise no error at all.

```
The average coupon is 1.875
```

Handle bad data

Report the bad value and keep going.

```
except ValueError:
```

Four bonds and a bug in every cell

Ticker	Coupon	Price	Rating	Face held
QVNE	5.25	98.50	BBB	1,500,000
DNMR	6.125	103.25	BB+	500,000
PLWK	4.75	95.00	A-	750,000
HLVF	7.50	101.00	B+	250,000

The new analyst's code

Written for these four bonds. Every piece of it has a bug.

The red error blocks are there on purpose.

The same four steps for every bug

1

Run it

and let it fail

→

2

Read it

from the last line up

→

3

Find the cause

work out why

→

4

Fix it

and run it again

Read a traceback from the bottom up

```
NameError        Traceback (most recent call last)
Cell In[3], line 4
      1 # store the coupon of QVNE
      2 coupon = 5.25
      3 # print it
----> 4 print('The coupon is', cupon)

NameError: name 'cupon' is not defined
```

3 **The top**

Which cell the error is in

2 **The arrow**

The line where Python stopped

1 **The last line: start here**

The error type and the message

Stored as `coupon` Typed as `cupon` To Python those are two different names.

The moth in the Mark II

1878

Thomas Edison used the word bug for a fault in a letter. Engineers already used the word that way.

September 9, 1947

Engineers working on Harvard's Mark II computer found a moth inside the machine. Someone taped it into the log book with a note calling it the first actual case of a bug being found.

Today

The log book, moth included, is in the Smithsonian's National Museum of American History.

Often told about Grace Hopper

She worked on the Mark II. The museum notes the log book was probably not hers.

`bug`	A mistake in code
`debugging`	Finding and fixing it

Source: JSTOR Daily, The Bug in the Computer Bug Story; Smithsonian National Museum of American History.

Four mistakes Python cannot read past

A missing colon

```
for ticker in ticker_list
    print(ticker)
SyntaxError: expected ':'
```

= where == was needed

```
if price = 100:
    print('QVNE trades at par')
SyntaxError: invalid syntax. Maybe you meant
'==' or ':=' instead of '='?
```

A bracket never closed

```
new_coupon_list = [5.25, 6.125, 4.75, 7.5, 6.0
print(new_coupon_list)
SyntaxError: '[' was never closed
```

A line not indented

```
for price in price_list:
print(price)
IndentationError: expected an indented block
after 'for' statement on line 1
```

A syntax error means Python could not read the code. Nothing in the cell runs.

The code starts running, then stops

```
bond = {Ticker: 'QVNE', 'Coupon': 5.25, 'Price': 98.5}
NameError: name 'Ticker' is not defined
```

NameError

Without quotes, Python reads Ticker as a variable name.

```
description = 'QVNE ' + 5.25 + '% of ' + 2033
TypeError: can only concatenate str (not "float") to str
```

TypeError

The operation is fine. The data type is wrong for it.

```
face = '1500000'
income = face * coupon / 100
TypeError: can't multiply sequence by non-int of type 'float'
```

type()

When the message makes no sense, check the type of each variable.

Asking for something that is not there

index	0	1	2	3	4
`ticker_list`	`'QVNE'`	`'DNMR'`	`'PLWK'`	`'HLVF'`	`IndexError`
`range(1, 5)`	*skipped*	1	2	3	4
`range(4)`	0	1	2	3	

```
bond = {'Ticker': 'QVNE', 'Coupon': 5.25,
        'Price': 98.5}
print(bond['coupon'])
KeyError: 'coupon'
```

Keys must match exactly

The key is stored as 'Coupon', with a capital C. Print bond.keys() to see how the keys are spelled.

When the data is the problem

ValueError: right type, wrong content

```
coupon_text = '5.25%'
coupon = float(coupon_text)
ValueError: could not convert string to float:
'5.25%'
```

↓

```
coupon = float(coupon_text.replace('%', ''))
5.25
```

Replace the percent sign with nothing, then convert.

ZeroDivisionError: right code, wrong data

```
feed_price_list = [98.5, 103.25, 0, 101.0]
QVNE 5.33
DNMR 5.93
ZeroDivisionError: float division by zero
```

↓

```
if price <= 0:
    print(ticker, 'has a bad price:', price)
else:
    print(ticker,
          round(coupon / price * 100, 2))
```

Deal with the bad value and report it. Leave the formula alone.

An error inside a function: two arrows

```
Cell In[37], line 5
----> 5     print(ticker, round(current_yield(coupon, price), 2))

Cell In[36], line 6, in current_yield(coupon, price)
----> 6     return coupon / price * 100

TypeError: unsupported operand type(s) for /: 'float' and 'str'
```

Upper arrow

Our loop. How Python got there.

Lower arrow

In the function. Where it failed.

Last line

Cannot divide a float by a string.

One price arrives in the feed as text: '95.0' → The loop hands it to current_yield → The division fails inside the function

The function is fine. It was handed bad data. The fix belongs in the loop.

Every error so far, in one table

Error	What Python is telling you	First thing to check
SyntaxError, IndentationError	The code is not in a form Python can read	A missing colon, bracket, or quote; an unindented line
NameError	A name has not been created	Spelling, a missing quote, a missing import, a cell not yet run
TypeError	The operation does not work on this type of data	type() of each variable; text where a number belongs
ValueError	The type is right, the content is not	The text being converted, such as '5.25%'
IndexError	The position is past the end of the list	The length of the list; a range that is one off
KeyError	The dictionary has no such key	The keys, spelled and capitalised exactly
AttributeError	This type of object has no such method	The method name, with dir() or help()
ZeroDivisionError	A division by 0	The data: a bad zero is usually the problem

Read the last line of the traceback first. Find its name here. Check the first thing in the right-hand column.

It runs, and the number is wrong

Annual coupon income on QVNE

```
coupon = 5.25
income = face * coupon
The annual coupon income is 7875000.0
```

Expected near 79,000: a little over 5 percent of the face

Simple average coupon of the four bonds

```
for coupon in coupon_list:
    total = coupon
The average coupon is 1.875
```

Expected between 4.75 and 7.5, the range of the coupons

Cannot see the cause? Add a print() inside the loop and watch the values change.

Three more	What happens	The habit	In Excel
Two names, one list	what_if_list = price_list makes no copy. A change shows under both names.	what_if_list = price_list.copy()	A copied range is a separate set of cells.
A borrowed name	After sum = 0, the call sum(coupon_list) fails: 'float' object is not callable.	Keep sum, max, min, len for Python. del sum undoes it.	
Decimals	0.1 + 0.2 is 0.30000000000000004, so == 0.3 is False.	Compare with math.isclose(). Show with round().	=0.1+0.2=0.3 is TRUE.

One bad value should not stop the run

try and except

```
for ticker, text in zip(ticker_list,
                        feed_text_list):
    try:
        price = float(text)
        print(ticker, 'is priced at', price)
    except ValueError:
        print(ticker,
              'has a price that is not a number:',
              text)
```

Always name the error you expect. A bare except: catches every error, your own typos included.

Excel side by side

Excel shows	Python raises
#NAME?	NameError
#VALUE!	TypeError or ValueError
#DIV/0!	ZeroDivisionError
#REF!	IndexError
#N/A	KeyError

IFERROR plays the role of try and except, but it catches every error. IFNA catches only #N/A. Excel puts the error in one cell and carries on. Python stops and tells you the line.

Checks that stop

```
assert condition, 'message to show if the condition is False'
AssertionError: expected 3,000,000 of face and found 3,250,000
```

assert

For your own work. If the condition is false, the run stops with your message.

```
def current_yield(coupon, price):
    if price <= 0:
        raise ValueError(
            f'the price must be above zero, and it is {price}')
    return coupon / price * 100
ValueError: the price must be above zero, and it is 0
```

raise

Inside a function. Refuse a value the function cannot use, and say why.

```
try:
    print(f'{ticker}: {current_yield(coupon, price):.2f} percent')
except ValueError as problem:
    print(f'{ticker}: skipped, {problem}')
DNMR: skipped, the price must be above zero, and it is 0
```

try / except

In the code that calls it. Decide what to do, and report the bad value.

The rule from here on: a check that matters is an assert, not a printed number. An Excel check cell shows its result while every other formula carries on. An assert stops everything below it.

The AI check

Every notebook ends with code written in the style of an AI assistant. It has one bug. Find it.

1

Read

Read the code before you run it.

2

Predict

Decide what it should produce.

3

Run

Run it and look at the output.

4

Check

Compare with a number you worked out yourself.

Code that runs is not the same as code that is right.

What you can do now

- Read a traceback: the error type, the message, and the line
- Name the common errors and say what each one is telling you
- Follow an error inside a function back to the line that caused it
- Catch a wrong number that raises no error
- Stop a run with assert, and handle bad data with try and except

Tomorrow Classes: a bond that carries its own data and calculations